

Problem Solving: From Fire-Fighting to Systematic Practice

SEAtS | 661057

Intended Learning Outcomes

By the end of this lecture, you will be able to:

- Distinguish intuitive (fire-fighting) problem-solving from systematic problem-solving, and explain why organisations default to the former.
- Apply a general-purpose problem-solving process that holds across software engineering, physics/engineering, and management contexts.
- Select an appropriate problem-solving method — general-purpose or systematic-inventive — for a given type of problem.
- Compare unsystematic idea-generating techniques (e.g. brainstorming) with systematic inventive-thinking methods (TRIZ, SIT/ASIT, SCAMPER).
- Identify the organisational and environmental conditions that determine whether systematic problem-solving is adopted and sustained.

Agenda

- Two Modes of Problem–Solving
- The Universal Process
- Ten General–Purpose Methods
- From Brainstorming to TRIZ
- Worked Example: Rotate List
- Engineering Practice
- Why Systematic Problem–Solving Doesn't Stick
- Conclusion



Video player configuration error

Error 153

Watch video on YouTube

[Learn more](#)

Error 153

Two Modes of Problem–Solving

✗ Intuitive Problem–Solving (IPS)

"System 1" — minimal cognitive effort, short-term remedies and heuristics that temporarily fix the *symptom*. Fast, but leads to "severe and systematic errors" and fails to prevent recurrence.

✓ Systematic Problem–Solving (SPS)

"System 2" — deliberative, analytical effort that diagnoses and alters the *underlying cause*. Slower and more resource-intensive, but produces robust, non-recurring solutions.

Despite near-universal agreement that SPS is superior, organisations repeatedly default to IPS — the **fire-fighting syndrome**. Part IV comes back to why.

1. Mohaghegh, M. and Furlan, A. (2020). Systematic problem-solving and its antecedents: a synthesis of the literature. *Management Research Review*, 43(9), pp. 1033–1062.

Intuitive Problem–Solving (IPS)

Ten General–Purpose Methods

Trial and Error

Keep testing until something works — good for hands-on problems with no single right answer.

Breaking It Down

Split a problem into smaller parts so the whole feels doable.

Root Cause Analysis

Keep asking "why" until you reach the real source, not the symptom.

Brainstorming

Generate ideas freely, without filtering — works best in groups, for creative problems.

Working Backwards

Start from the end result and retrace the steps needed to get there.

Lateral Thinking

Abandon the obvious path; approach from an unconventional angle.

SWOT Analysis

Strengths, Weaknesses, Opportunities, Threats — a four-angle decision framework.

Mind Mapping

Visual map from a central idea, branching into topics and sub-topics.

Heuristic Method

Quick, "good enough" solutions from experience and rules of thumb.

Visualization

Mentally rehearse solving the problem before acting.

These aren't mutually exclusive — the best solutions often combine several (e.g. break the problem down, then run root cause analysis on one sub-problem).

The Universal Process

The same loop reappears whether you're writing code, solving a physics problem, or running a business — only the vocabulary changes.

Step	Software Engineering	Physics / Engineering	Management (SPS)
Understand	Write down what you know; ask about inputs/outputs	<i>Thinking</i> : read carefully, identify given vs. needed	Problem definition
Devise a strategy	Divide and conquer; reuse known algorithms	<i>Thinking</i> : sketch the system, strategize	Problem analysis (root cause)
Carry it out	Combine sub-solutions	<i>Doing</i> : carry out the strategy with symbols	Solution design & evaluation
Check	Due diligence — test edge/corner cases	<i>Doing</i> : check the answer makes sense	Solution selection
Lock it in	Optimize using known algorithms/data structures	—	Standardise (PDCA "Act" / DMAIC "Control") — prevents recurrence

1. Video notes, Florian (software engineering); "R2 — Engineering Problem Solving Skills" (physics/engineering); Mohaghegh & Furlan (2020).

Example: Rotate List

Step 1 – Understand

Given the head of a linked list, rotate the list to the right by `k` places.

Clarifying questions, as in a real interview: Is there a maximum length? Could the list be empty, or have one element? Can `k` be negative?

Reframe: rotating the last `k` elements to the front $\equiv$ rotating the first `(length - k)` elements to the back.

Reduce: if `k` exceeds the list length, only `k mod length` rotations matter.

× First Attempt Fails

Move the tail to the front — but a singly linked list only has `next` pointers, not `prev`.

Deriving the Algorithm

Step 2 – Devise a strategy

Working approach: move front elements to the back instead — this only needs `head` and the known `tail`.

1. Find the list length (the same traversal locates the tail).
2. Simplify: `true_k = length - (k mod length)`.
3. Connect `tail.next = head` (temporary circular list); walk `true_k` steps from `head`; disconnect `.next` there; return the next node as the new head.

Edge cases: empty list → return as-is. Single-element list → return as-is (would otherwise self-reference).

Step 3 – Carry it out

```
def rotateRight(head, k):  
    if head is None:  
        return head  
  
    length = 1  
    current = head  
    while current.next is not None:  
        length += 1  
        current = current.next  
  
    if length == 1:  
        return head  
  
    true_k = length - (k % length)  
    current.next = head          # temporary circular list  
  
    current = head  
    for _ in range(true_k - 1):  
        current = current.next  
  
    new_head = current.next  
    current.next = None  
    return new_head
```

Bugs Found by Tracing Through by Hand

Step 4 – Check

- `length` started at `1`, not `0` — the traversal loop already assumes one node exists.
- The rotation loop ran `true_k - 1` times, not `true_k` — one "step" is already accounted for by starting `current` at `head`.

Step 5 – Lock it up

🔥 Takeaway

This mirrors the Universal Process exactly: understand (clarifying questions) → devise a strategy (whiteboard reasoning) → carry it out (implementation) → check (manually tracing before running). Manually tracing code before running it is a habit worth keeping outside interviews too.

From IPS to SPS

Most of the methods just covered are **unsystematic idea-generating**: free association, free of judgement, triggering imagination. They feel creative — but the evidence is mixed.

❓ Does Brainstorming Actually Work?

"Research indicates that most brainstorming groups did not generate more ideas than control groups... The quality of the ideas themselves and their level of originality were inferior to the ideas generated by individuals working without any group effect." — Goldenberg & Mazursky

A creative solution doesn't require a large number of ideas. **Focused thinking and early judgement, against well-defined criteria, can out-perform free-association.** That's the case for *systematic* inventive thinking.

1. Barak, M. (2004). Systematic Approaches for Inventive Thinking and Problem-Solving: Implications for Engineering Education. *Int. J. Engng Ed.*, 20(4), pp. 612–618.

TRIZ — Theory of Inventive Problem Solving

Developed by Altshuller from a study of over a million patents, TRIZ is based on three principles:

1. Resolving technical and physical contradictions
2. The evolution of systems
3. Reference to the ideal system and ideal solution

These generate **40 "Techniques for Overcoming System Conflicts"** (e.g. Segmentation, Asymmetry, Universality, Nesting). Used at Motorola, Xerox, Kodak, Ford, GE, and Rolls–Royce.

SIT / ASIT — Five Thinking Tools

A simplification of TRIZ, governed by the **Closed World (CW) principle**: a creative solution relies on components *already in the system* — adding new elements usually isn't considered creative.

Tool	What it does
Unification	Assign a new function to an existing component
Multiplication	Introduce a (modified) copy of an existing object
Division	Divide an object and reconfigure its parts
Object Removal	Remove an object from the system
Breaking Symmetry	Change symmetrical/asymmetrical relationships

The Closed World Principle states that when solving a problem or innovating, you must use only elements and resources that already exist within the system or its immediate environment.

Example: The Curved Pipe (Unification)

A corn-processing plant's curved pipe erodes where an airstream propels grains around the elbow. List the Closed World: **pipe, grains, air**.

Formulate the Unification sentence: *"A XXX component from the CW will be used to prevent the grains from eroding the pipe."* Substitute each CW component for XXX:

- A **pipe** will prevent the grains from eroding the pipe — the conventional answer (reinforce the pipe).
- The **grains** will prevent the grains from eroding the pipe — *how?*

✓ The Systematic Answer

Reshape the pipe so trapped grains themselves form a protective lining — the grains protect the pipe from the grains.

Vevox: The Five Glasses

☰ Try It — Using One of the Five Thinking Tools

Six glasses stand in a row; the three on the left are full. Moving only **one** glass, alternate full and empty.

Now solve the **five**-glasses version yourself: rearrange them to alternate empty/full, moving only one glass. Which tool did you use?



SCAMPER

An earlier, broader checklist of prompts (Osborn; acronym by Eberle) for reworking an existing idea or object:

Substitute · **C**ombine · **A**dapt · **M**agnify · **M**odify · **P**ut to other uses · **E**liminate (or divide) · **R**everse

Kaplan (reporting Horowitz) summarises the relationship simply: **ASIT $\approx$ SCAMPER + the Closed World condition.**

The Eight–Dimensional Methodology

Raviv's framework, taught at Florida Atlantic University through hands–on puzzles, games, and design projects:

Uniqueness · Dimensionality · Directionality · Consolidation

Segmentation · Modification · Similarity · Experimentation

Each dimension is a question you can ask of *any* problem — e.g. "Could this be done from a different direction?" (Directionality) or "What if we experimented/simulated instead of calculating?" (Experimentation).

Seven Factors That Support SPS Adoption

A systematic literature review (93 papers, via co-citation analysis) identifies seven recurring factors:

Factor	What it captures
Nature of the problem	Well-structured problems tolerate IPS; ill-structured ones need SPS
Time availability	SPS pays off with a delay — urgency pushes people back to fire-fighting
Information availability	Rich data on the problem's context enables SPS; "insist on data, not assumptions"
Collaborative culture	Information sharing across units facilitates SPS
Transformational leadership	Leaders who tolerate the short-term cost of SPS sustain its adoption
Organizational learning infrastructure	Mechanisms to capture and reuse what was learned
Environmental dynamism	Stable environments support SPS; volatile ones push back toward IPS

1. Mohaghegh & Furlan (2020), synthesising 93 co-cited articles into these seven second-order themes.

Towards an Integrative Model

Uncertainty is the core mechanism. Ill-structured problems carry *internal* uncertainty (the problem's own complexity); volatile environments add *external* uncertainty.

SPS adoption requires organisations to actively **reduce both kinds of uncertainty** — through information, collaboration, learning infrastructure, and leadership that can absorb the short-term cost of not just fire-fighting.

Cognitive Debiasing

This is why the seven factors matter: they're not a checklist, they're the conditions under which deliberate, System-2 analysis can actually out-compete the fast, heuristic defaults of System 1.

Conclusion

- Systematic problem-solving (SPS) consistently out-performs intuitive fire-fighting (IPS) — but IPS is the default organisations drift back to.
- The same four/five-step process — understand, strategize, execute, check, lock in — holds across software, physics/engineering, and management.
- Systematic *inventive* thinking (TRIZ, SIT/ASIT, SCAMPER, Eight-Dimensional) out-performs unsystematic idea-generating (brainstorming) precisely because it focuses and judges early rather than deferring judgement.
- Whether SPS is actually *adopted and sustained* depends on organisational and environmental conditions, not just knowing the right method.
- This mirrors L3.1's systemic-vs-systematic distinction: a method alone doesn't guarantee systematic practice — the surrounding system has to support it.

Thank you

Dr Nawfal Al Hashimy

Lecturer in Cyber Physical Security

University of Southampton | Faculty of Engineering and Physical
Sciences

Cyber Physical Systems Research Group

☎ +44 (0) 23 8059 6659

@ Nawfal@soton.ac.uk

Highfield Campus, Building 53, Room 4049
Burgess Road, Southampton SO17 1BJ, UK

